How Elixir Implements If Statements



Elixir implements if statements as a macro that expands to a single case statement. You can see this in the Elixir Kernel source.[1]

This means that if statements are a subset of case statements. A case statement can do everything that an if statement in Elixir can do, but it can do additional things as well.

Elixir doesn't provide an else if syntax. You can nest if statements to create this type of comparison waterfall, but it quickly becomes messy. Instead, case and cond statements are well-suited for checking multiple potential match values. Let's look at case statements now.

Case Statements

A case statement evaluates a given term (any piece of data) against multiple potential patterns. The first pattern that matches the term executes a given block of code.

Case statements are the most commonly used and foundational control flow structure. Use case to check a single term against a number of potential patterns. The first clause that matches will have its body executed and returned.

The next code example demonstrates a case statement with multiple clauses:

```
$ iex
iex>
case {:ok, "a string"} do
  :not_a_match ->
    IO.puts "will not run"

  {:ok, string} ->
    IO.puts(string)
    "return value"
end

a string
"return value"
```

This case statement evaluates each pattern against the given term {:ok, "a string"}. The first term doesn't match, so its code is ignored. The second term does match, so the code is executed and its value is returned.

Case statements are meant to be exhaustive. This means that every possible outcome should be checked against, or else you'll receive a CaseClauseError:

1. https://github.com/elixir-lang/elixir/blob/v1.14.1/lib/elixir/lib/kernel.ex#L3720